

Trabalho de Álgebra Linear Computacional 26.1

Decomposição em Valores Singulares na Adaptação de Baixo Posto de Modelos de Linguagem

Vitoria Leite¹

¹ Instituto de Computação – Universidade Federal Fluminense (UFF)
Niterói – RJ – Brasil – 24210-346

Abstract. *Este trabalho propõe um estudo da Decomposição em Valores Singulares (SVD) e de sua aplicação em aproximações matriciais de baixo posto. Inicialmente, serão apresentados os fundamentos teóricos da SVD, destacando o papel dos valores singulares na identificação das direções de maior relevância estrutural dos dados e sua relação com o problema de melhor aproximação de posto k . Em seguida, o trabalho conectará essa perspectiva matemática ao contexto de Modelos de Linguagem Grandes (LLMs), analisando o desafio computacional e de memória exigido pelo fine-tuning tradicional. Para superar esse gargalo, será detalhado o método Low-Rank Adaptation (LoRA), que congela os pesos originais e injeta matrizes de atualização de baixo posto nas camadas do modelo. Também serão citadas extensões avançadas presentes na literatura, como o AdaLoRA, que aloca o orçamento de parâmetros de forma adaptativa com base na importância das matrizes. Por fim, será desenvolvida uma implementação computacional prática de cada método, avaliando o trade-off entre a escolha do posto da matriz, a drástica redução de parâmetros treináveis e a preservação do desempenho do processo de fine-tuning.*

Keywords: fine tuning, LLM, SVD, aproximação matricial de baixo posto.

1. Introdução

A Decomposição em Valores Singulares (SVD) consolidou-se como uma ferramenta matemática poderosa pela sua capacidade de extrair a estrutura essencial de matrizes através da aproximação de baixo posto. Nos últimos anos, essa propriedade encontrou uma aplicação de destaque no campo da Inteligência Artificial, mais especificamente na adaptação de Modelos de Linguagem Grandes (LLMs). O ajuste fino completo desses modelos, que envolve o recálculo e a atualização de matrizes de peso gigantescas, tornou-se um gargalo proibitivo em termos de memória e processamento de hardware.

A literatura recente demonstrou que modelos pré-treinados possuem uma dimensionalidade intrínseca muito baixa [Aghajanyan et al. (2020)], ou seja, existe uma reparametrização de baixa dimensão que é tão eficaz para o aprendizado de novas tarefas quanto o espaço completo de parâmetros.

Baseando-se nesse princípio, o método Low-Rank Adaptation (LoRA) [Hu et al. (2021)] propôs o congelamento da matriz original e a injeção de matrizes de atualização parametrizadas como o produto de duas matrizes menores de posto reduzido (BA), atuando não apenas como uma forma de reduzir custo computacional, mas como um regularizador estrutural. Entretanto, o método LoRA distribui esse posto de forma estática e uniforme por todas as camadas, ignorando a assimetria natural de importância que cada matriz exerce na rede neural.

Para solucionar essa ineficiência na distribuição, o método Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning (AdaLoRA) [Zhang et al. (2023)] foi proposto. O AdaLoRA mimetiza ativamente a Decomposição em Valores Singulares em tempo de treinamento, forçando penalidades de ortogonalidade matricial e podando os valores singulares de menor relevância baseando-se em métricas de importância calculadas para cada camada.

Este trabalho propõe uma análise do Full Fine-Tuning, LoRA e AdaLoRA sob a ótica da álgebra linear. A partir do treinamento do modelo GPT-2 no dataset estruturado WikiText-2, foi construído um ambiente comparativo que garanta que as técnicas modernas de ajuste fino por aproximação obedecessem aproximadamente ao mesmo limite final de parâmetros treináveis.

O objetivo é investigar, através da extração das matrizes resultantes e da análise de métricas, como a retenção de energia da SVD, como a restrição de posto das atualizações de pesos afeta o custo computacional, a quantidade de parâmetros treináveis e o comportamento da perda durante o ajuste fino de um modelo de linguagem.

2. Fundamentação teórica

Para começar a explorar o método aqui apresentado, é necessário solidificar alguns conceitos e métodos importantes no campo da álgebra linear que servem de fundação teórica para o trabalho.

A primeira ideia importante para pensar em algoritmos computacionais baseados em álgebra linear é a decomposição de matrizes. Se operações matrizes-vetor podem ser vistas como formas de representar transformações lineares, queremos sempre que possível melhorar a performance dessas operações. Uma forma de atacar esse ponto é analisar formas interessantes de decomposição de uma matriz que resulte em economia operacional e melhor performance computacional.

2.1. Decomposição em autovalores (EVD)

Nesse cenário, vamos rapidamente retomar a decomposição EVD: a partir dos autovalores e autovetores de uma matriz.

Se temos uma matriz A quadrada $\in \mathbb{R}^{n \times n}$ que representa uma transformação linear $A : V \rightarrow V$ entre espaços vetoriais V , um λ é dito autovalor dessa matriz se existe um vetor diferente do vetor nulo tal que

$$Ax = \lambda x$$

Podemos entender, pela intuição geométrica, que existe um x que quando impactado por essa transformação descrita pela matriz A tem o efeito apenas de resultar em um vetor múltiplo de seu vetor original. Esse x chamamos de autovetor.

Esse achado é importante para o campo da álgebra computacional pois os autovalores de A são números que destacam as informações essenciais daquela matriz, o que nos dá a motivação de decompor a matriz A se utilizando apenas do que A é essencial. Supondo portanto que a matriz A tem

n autovetores linearmente independentes q_i (onde $i = 1, \dots, n$), podemos escrever:

$$\begin{aligned} Aq_i &= \lambda_i q_i & \forall i \in 1, \dots, n \\ Aq_i &= \lambda_i Iq_i & \forall i \in 1, \dots, n \\ Aq_1 + \dots + Aq_n &= \lambda_1 Iq_1 + \dots + \lambda_n Iq_n \\ AQ &= Q\Lambda \\ A &= Q\Lambda Q^{-1} \end{aligned}$$

Onde Q é a matriz cuja coluna i é q_i e Λ é a matriz cujo elemento diagonal i é λ_i . Essa é chamada decomposição em autovalores e, apesar de ser bastante intuitiva, requer que A seja uma matriz quadrada com número suficiente de autovetores linearmente independentes. Portanto, seu campo de possibilidade de aplicações é bastante limitado.

2.2. Decomposição em Valores Singulares (SVD)

Como generalização da ideia da EVD, nasce a SVD admitindo qualquer matriz A . Supondo agora uma matriz $A \in \mathbb{R}^{m \times n}$ qualquer, temos que AA^T e $A^T A$ serão, por definição, matrizes quadradas e simétricas que possuem autovalores reais não-negativos $\lambda_i \geq 0$ e um conjunto de autovetores ortonormais v_i . Logo, podemos escrever que:

$$A^T A v_i = \lambda v_i$$

Analisando de forma mais abstrata a multiplicação Av_i , temos um vetor resultado que é definido por uma direção e uma norma. Se calcularmos a norma quadrada de Av_i temos:

$$\begin{aligned} \|Av_i\|_2^2 &= (Av_i)^T (Av_i) \\ \implies \|Av_i\|_2^2 &= v_i^T A^T A v_i \\ \implies \|Av_i\|_2^2 &= v_i^T \lambda_i v_i \\ \implies \|Av_i\|_2^2 &= \lambda_i (v_i^T v_i) \end{aligned}$$

Como v_i são vetores unitários,

$$\implies \|Av_i\|_2^2 = \lambda_i$$

Logo, definimos os valores singulares de A como $\sigma_i = \sqrt{\lambda_i}$. Para os índices tais que $\sigma_i > 0$, podemos calcular o vetor unitário na direção de Av_i como u_i , tal que:

$$u_i \sigma_i = Av_i \tag{1}$$

$$u_i = \frac{1}{\sigma_i} Av_i. \tag{2}$$

Quando $\sigma_i = 0$, temos $\lambda_i = 0$ e, como $\|Av_i\|_2^2 = \lambda_i$, segue que $Av_i = 0$. Nesse caso, não definimos u_i

pela expressão acima, pois não existe uma direção não nula a ser normalizada.

Para os vetores associados a valores singulares não nulos, ainda precisamos verificar se os vetores u_i também formam um conjunto ortonormal. Tomando $\sigma_i, \sigma_j > 0$, temos:

$$\begin{aligned} u_i^T u_j &= \left(\frac{1}{\sigma_i} A v_i\right)^T \left(\frac{1}{\sigma_j} A v_j\right) \\ \implies u_i^T u_j &= \frac{1}{\sigma_i \sigma_j} v_i^T (A^T A v_j) \\ \implies u_i^T u_j &= \frac{\sigma_j}{\sigma_i} v_i^T v_j \end{aligned}$$

Como v_i e v_j são ortonormais, temos que $v_i^T v_j = 0$ se $i \neq j$ e $v_i^T v_j = 1$ se $i = j$. Portanto, os vetores u_i associados aos valores singulares não nulos são ortonormais. Caso seja necessário completar U como uma matriz ortogonal de dimensão $m \times m$, os vetores restantes podem ser escolhidos de modo a completar esse conjunto para uma base ortonormal de $\mathbb{R}^m$.

Com isso, podemos utilizar (1) empacotando para a forma matricial onde Σ é a matriz diagonal cujas entradas são os valores singulares e U a matriz ortonormal com os vetores u_i , temos que:

$$\begin{aligned} AV &= U\Sigma \\ \implies A &= U\Sigma V^{-1} \\ \implies A &= U\Sigma V^T \end{aligned}$$

Essa é a chamada Decomposição em Valores Singulares. Como U e V são matrizes ortogonais, temos $U^{-1} = U^T$ e $V^{-1} = V^T$. Geometricamente, matrizes ortogonais preservam comprimentos e ângulos, podendo ser interpretadas como rotações possivelmente acompanhadas de reflexões.

Assim, a transformação dada por A pode ser entendida em três etapas: primeiramente, V^T muda o sistema de coordenadas de entrada para a base formada pelos vetores singulares direitos v_i . Em seguida, Σ escala cada uma dessas direções pelo respectivo valor singular σ_i . Por fim, U leva o resultado para a base formada pelos vetores singulares esquerdos u_i , definindo a orientação final da transformação.

2.3. Aproximação de posto k

Com essa decomposição em mente e a ideia de que os autovalores destacam parte importante das propriedades das matrizes, podemos tentar analisar uma matriz suprimida A_k construída mantendo os k maiores valores singulares e verificar o quanto conseguimos continuar mantendo suas propriedades.

Computacionalmente, é interessante poder pensar na maior compressão possível que preserve os atributos importantes da matriz para que processos complexos sejam simplificados, oferecendo ganho de tempo de execução e menor uso de recursos.

Dessa forma, dada a decomposição SVD de uma matriz A com posto r , definimos uma matriz

aproximada de posto k como sendo:

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$$

Onde convencionamos que $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n$.

Para verificar que essa matriz truncada é a aproximação mais fiel da matriz original dentre todas as matrizes de posto k, calculamos o erro que inserimos ao fazer essa substituição utilizando a norma-2

$$\begin{aligned} A - A_k &= \sum_{i=1}^n \sigma_i u_i v_i^T - \sum_{i=1}^k \sigma_i u_i v_i^T \\ \implies A - A_k &= \sum_{i=k+1}^n \sigma_i u_i v_i^T \\ \implies \|A - A_k\|_2 &= \left\| \sum_{i=k+1}^n \sigma_i u_i v_i^T \right\|_2 \\ \implies \|A - A_k\|_2 &= \sigma_{k+1} \end{aligned}$$

Agora, supomos a existência de uma matriz $B \in \mathbb{R}^{m \times n}$ de posto k. Como a matriz tem posto k, sabemos que $\dim(\ker(B)) = (n - k)$.

Tomando $\mathcal{V}_{k+1}$ como o espaço vetorial formado pelos primeiros $k + 1$ vetores singulares de A, temos que $\mathcal{V}_{k+1} = \text{span}\{v_1, v_2, \dots, v_{k+1}\}$ tem dimensão $(k + 1)$.

Se somarmos a dimensão de $\ker(B)$ e $\mathcal{V}_{k+1}$ obtemos:

$$\dim(\ker(B) + \mathcal{V}_{k+1}) = \dim(\ker(B)) + \dim(\mathcal{V}_{k+1}) - \dim(\ker(B) \cap \mathcal{V}_{k+1})$$

Como ambas são subespaços de $\mathbb{R}^n$, temos:

$$\begin{aligned} \dim(\ker(B) + \mathcal{V}_{k+1}) &\leq n \\ \implies \dim(\ker(B)) + \dim(\mathcal{V}_{k+1}) - \dim(\ker(B) \cap \mathcal{V}_{k+1}) &\leq n \\ \implies \dim(\ker(B) \cap \mathcal{V}_{k+1}) &\geq \dim(\ker(B)) + \dim(\mathcal{V}_{k+1}) - n \\ \implies \dim(\ker(B) \cap \mathcal{V}_{k+1}) &\geq (n - k) + (k + 1) - n \\ \implies \dim(\ker(B) \cap \mathcal{V}_{k+1}) &\geq 1 \end{aligned}$$

O que significa que a interseção entre os dois espaços é não trivial.

Com isso em mente, podemos tomar um $z \in \ker(B) \cap V_{k+1}$ tal que $\|z\|_2 = 1$, dessa forma:

$$Bz = 0$$

$$z = \sum_{i=1}^{k+1} \gamma_i v_i \quad \text{onde} \quad \gamma_1^2 + \dots + \gamma_{k+1}^2 = 1$$

Com isso, podemos calcular a distância entre A e B como:

$$\begin{aligned} \|A - B\|_2 &= \|A - B\|_2 \cdot \|z\|_2 \\ \|A - B\|_2 \cdot \|z\|_2 &\geq \|(A - B)z\|_2 \\ \implies \|A - B\|_2 &\geq \|(A - B)z\|_2 \\ \implies \|A - B\|_2^2 &\geq \|(A - B)z\|_2^2 \\ \implies \|A - B\|_2^2 &\geq \|Az - Bz\|_2^2 \\ \implies \|A - B\|_2^2 &\geq \|Az\|_2^2 \end{aligned}$$

Agora calculamos Az :

$$\begin{aligned} Az &= A\left(\sum_{i=1}^{k+1} \gamma_i v_i\right) \\ \implies Az &= \sum_{i=1}^{k+1} \gamma_i (Av_i) \\ \implies Az &= \sum_{i=1}^{k+1} \gamma_i \sigma_i u_i \\ \implies \|Az\|_2^2 &= \sum_{i=1}^{k+1} \sigma_i^2 \gamma_i^2 \\ \implies \|Az\|_2^2 &\geq \sigma_{k+1}^2 (\gamma_1^2 + \dots + \gamma_{k+1}^2) \\ \implies \|Az\|_2^2 &\geq \sigma_{k+1}^2 \end{aligned}$$

Portanto,

$$\begin{aligned} \|A - B\|_2^2 &\geq \sigma_{k+1}^2 \\ \implies \|A - B\|_2 &\geq \sigma_{k+1} \end{aligned}$$

Provamos então que essa matriz suprimida é garantida dentre todas as matrizes de posto k como a que mais se aproxima da nossa matriz original no que diz respeito a diferença de norma-2. Esse teorema é chamado de *Eckart-Young-Mirsky* [\[Eckart and Young \(1936\)\]](#).

Ou seja, caso tenhamos necessidade computacional de redução de dimensionalidade, a matriz de posto k calculada a partir da decomposição SVD é a melhor substituição que podemos utilizar.

2.4. Finetuning de modelos LLMs

No contexto atual, encontramos alguns modelos de linguagem enormes pré treinados em dados de domínio geral que garantem um certo nível de performance nas tarefas que são demandados. Para adapta-los para que sejam mais especialistas em um tipo de tarefa em específico, utilizamos o processo de ajuste fino onde buscamos dar para esses modelos generalistas uma versão com melhor performance para essas tarefas por meio de um recálculo dos pesos originais do modelo.

Entretanto, como esses modelos estão cada vez maiores, o processo de ajuste completo (re-treinamento de 100% dos parâmetros) fica cada vez mais custoso. Tomando como exemplo o *GPT-3*, que tem 175 bilhões de parâmetros, o custo em hardware e memória para criar e armazenar cada instância representativa da etapa de recálculo de parâmetros em um modelo desse porte é inviável.

Felizmente, como visto em [Aghajanyan et al. (2020)], existe uma reparametrização de baixa dimensão que é tão eficaz para o *fine-tuning* quanto operar no espaço de parâmetros completo e o próprio pré treinamento anterior da rede ajuda a minimizar essa chamada **dimensionalidade intrínseca**.

2.5. LoRA

É nesse cenário que surge a proposta do *Low-Rank Adaptation* (LoRA) [Hu et al. (2021)], uma técnica projetada para explorar essa característica dos LLMs e que se propõe a trabalhar, no caso do *GPT-3* por exemplo, com cerca de apenas 0.01% da quantidade total de parâmetros do modelo.

Em cada camada densa das redes neurais, computamos multiplicações matriciais no passo de propagação direta. A proposta seria que, dado um modelo pré treinado com uma matriz de pesos $W_0 \in \mathbb{R}^{d_1 \times d_2}$, representássemos esses passos de retreino usando uma decomposição $W_0 + \Delta W = W_0 + \frac{\alpha}{r}BA$, onde α é uma constante de escala, $B \in \mathbb{R}^{d_1 \times r}$, $A \in \mathbb{R}^{r \times d_2}$ e $r \ll \min(d_1, d_2)$.

Durante o fine-tuning, congelamos a matriz W_0 enquanto A e B são computadas como parâmetros treináveis, tornando a propagação direta do LoRA adaptada para:

$$h = W_0x + \Delta Wx = W_0x + \frac{\alpha}{r}BAx$$

No método LoRA tradicional, a inicialização da matriz A é feita a partir de uma gaussiana aleatória e B é a matriz nula e o posto r escolhido é o mesmo para todas as camadas da rede.

2.6. AdaLoRA

O AdaLoRA é uma atualização do LoRA que trabalha com a ideia de que cada camada da rede neural tem uma importância diferente para o aprendizado de um modelo e que tratar todas as camadas com um mesmo peso r dado pelo *rank* escolhido, leva a um desempenho subótimo desperdiçando a capacidade de adaptação em camadas menos relevantes e limitando camadas com maior necessidade de alocação de informação.

Para endereçar esse problema, o AdaLoRA calcula um score de importância entre as camadas durante as etapas do retreinamento e o utiliza como base para decidir qual *rank* utilizar em cada camada na etapa seguinte.

O método substitui a decomposição anterior de ΔW por uma decomposição profundamente inspirada em SVD onde $\Delta W = P\Lambda Q$ sendo $P \in \mathbb{R}^{d_1 \times r}$ e $Q \in \mathbb{R}^{r \times d_2}$ matrizes com colunas ortogonais e $\Lambda \in \mathbb{R}^{r \times r}$ uma matriz diagonal com os r valores singulares, onde $r \ll \min(d_1, d_2)$.

A propagação direta do AdaLoRA é então dada por:

$$h = W_0 x + \Delta W x = W_0 x + P \Lambda Q x$$

E, em cada passo, para garantir a ortogonalidade de P e Q, usa-se o regularizador:

$$R(P, Q) = \|P^T P - I\|_F^2 + \|Q Q^T - I\|_F^2$$

2.6.1. Alocação dinâmica e função de score

Cabe agora aprofundar na função de score que poda valores singulares de menor importância em cada etapa do treinamento de forma personalizada para cada camada. Temos na etapa (t) para a camada k a soma do incremento $\Delta_k^{(t)} = P_k^{(t)} \Lambda_k^{(t)} Q_k^{(t)}$ onde cada uma das matrizes é um parâmetro treinável utilizando o cálculo do gradiente.

Olhando especificamente para a matriz de valores singulares $\Lambda_k^{(t)}$ definimos $\tilde{\Lambda}_k^{(t)}$ como a matriz de valores singulares no passo (t) atualizada pelo gradiente, mas ainda não podada e $S_k^{(t)}$ como o resultado de uma função score da camada k na etapa (t) de forma que $S_{k,i}^{(t)}$ é particularmente o score do i -ésimo valor singular. Dessa forma, fazemos a poda usando uma função $\mathcal{T}(\tilde{\Lambda}_k^{(t)}, S_k^{(t)})$ dada por:

$$\mathcal{T}(\tilde{\Lambda}_k^{(t)}, S_k^{(t)})_{ii} = \begin{cases} \tilde{\Lambda}_{k,ii}^{(t)}, & \text{se } S_{k,i}^{(t)} \text{ está no top-}b^{(t)} \text{ de } S^{(t)} \\ 0, & \text{caso contrário} \end{cases}$$

E assim, sendo $b^{(t)}$ o orçamento previamente estabelecido de valores singulares para o passo (t) , mantemos apenas os valores singulares de maior score independente da camada que estejam e zeramos todo o restante.

A função score em si não possui uma definição rígida, mas aqui vamos utilizar a mesma função do artigo principal do AdaLoRA [Zhang et al. (2023)] e vamos definir s como:

$$s^{(t)}(w_{ij}) = \bar{I}^{(t)}(w_{ij}) \bar{U}^{(t)}(w_{ij})$$

onde $\bar{I}$ é a sensibilidade suavizada por média móvel exponencial, e $\bar{U}$ é o termo de incerteza quantificado pela variação local entre $I^{(t)}$ e $\bar{I}^{(t)}$. Ou seja, dados os parâmetros $1 > \beta_1, \beta_2 > 0$:

$$\begin{aligned} \bar{I}^{(t)}(w_{ij}) &= \beta_1 \bar{I}^{(t-1)}(w_{ij}) + (1 - \beta_1) I^{(t)}(w_{ij}) \\ \bar{U}^{(t)}(w_{ij}) &= \beta_2 \bar{U}^{(t-1)}(w_{ij}) + (1 - \beta_2) |I^{(t)}(w_{ij}) - \bar{I}^{(t)}(w_{ij})| \\ I^{(t)}(w_{ij}) &= |w_{ij} \Delta_{w_{ij}} \mathcal{L}| \end{aligned}$$

Onde $\Delta_{w_{ij}} \mathcal{L}$ é o gradiente da função de perda com relação a w_{ij} . Assim, S_k fica definido como:

$$S_{k,i}^{(t)} = s^{(t)}(\lambda_{k,i}) + \frac{1}{d_1} \sum_{j=1}^d s^{(t)}(P_{k,ji}) + \frac{1}{k} \sum_{j=1}^k s^{(t)}(Q_{k,ij})$$

3. Metodologia

Para avaliação da eficiência de cada uma das adaptações propostas, foram implementadas classes com as propriedades matemáticas aqui descritas que foram plugadas em um modelo pré-existente de forma a resguardar os pesos iniciais e permitir a iteração otimizada durante o ajuste fino.

Os testes foram feitos com o modelo GPT-2 pré-treinado disponibilizado pela lib *transformers* [Wolf et al. (2020)] onde as camadas lineares de attention e feed-forward foram substituídas (attn.c_attn, attn.c_proj, mlp.c_fc, mlp.c_proj) pelas classes criadas para cada proposta de adaptação. O dataset "WikiTextDataset" disponibilizado pela lib do hugging face [Merity et al. (2016)] foi o utilizado para o *fine-tuning* e o objetivo da implementação está mais relacionado à validação algébrica dos métodos do que de fato uma preocupação em ajustar um modelo de linguagem visando a maior eficiência em uma tarefa específica.

3.1. LoRA

A implementação do LoRA foi feita no arquivo *lora.py*, com a classe do layer inicializando as matrizes B como uma matriz de zeros e A como uma gaussiana aleatória. Foram mantidas essas como as únicas matrizes ajustáveis em cada camada, utilizando como parâmetro *requires_grad* = True e foi criado o método de forward cobrindo essas as matrizes de baixo posto inicializadas.

Foi também implementada uma classe para mesclar os pesos antigos com os novos calculados, para que ao final do treinamento seja possível obter o modelo com o ajuste total.

Durante as rodadas, o rank das matrizes foi fixado em 4 e foram utilizados como parâmetros um *weight decay* de 0.01, *learning rate* de $5e^{-5}$, $\alpha=r$ e o otimizador AdamW.

3.2. AdaLoRA

O AdaLoRA foi implementado no arquivo *adalora.py*, tendo a classe do layer inicializando as matrizes P e Q como matrizes gaussianas aleatórias e Λ como uma matriz de zeros. Assim como o LoRA, foram implementados os métodos de merge e forward adaptados, além de um método de cálculo da matriz de importância a ser feito dentro de cada layer.

Como a avaliação do budget de valores singulares e atualização da máscara de poda a ser aplicada em Λ deve ser feita entre todas as camadas do modelo, essas duas funções foram implementadas fora das classes, sendo chamadas a cada etapa de treinamento. Como o custo do cálculo da SVD é alto, a implementação se utiliza da aproximação da decomposição alcançada com a regularização descrita na seção anterior.

Para o cálculo da função de score utilizamos uma média exponencial controlada por parâmetros β_1 e β_2 (nas implementações ambos são definidos como 0.85) para maior estabilidade das matrizes. Os parâmetros *weight decay* e *learning rate* são iguais aos usados nos testes com LoRA e o otimizador utilizado foi novamente o AdamW.

Seguindo a implementação do artigo, o orçamento de valores singulares a ser utilizado inicia com um valor maior do que o valor alvo, permitindo o modelo ter maior liberdade para explorar o que é útil e eventualmente encontrar a melhor distribuição de *rank*, segundo sua função de score, e decai o valor de orçamento durante o processo de treino.

Esse decaimento de *rank* é governado por um agendador global de orçamento (*budget_scheduler*), que divide o treino em três etapas: inicialmente a rede opera com um *rank* 1.5 vezes maior que o valor

objetivo final e, em seguida, aplica-se um decaimento cúbico contínuo, evitando choques nas matrizes ortogonais durante as podas mais críticas. Por fim, o subespaço atinge o orçamento alvo, permitindo o refinamento final da função de custo. Essa atualização de orçamento e descarte de valores singulares não é feita a cada etapa de treinamento do modelo, mas sim a cada T passos parametrizável.

Para permitir que os métodos LoRA e AdaLoRA sejam comparáveis nas rodadas apresentadas, parametrizamos o orçamento final de valores singulares b_{final} aqui como sendo $b_{final} = 4 * n_{matrizes}$ onde $n_{matrizes}$ são a quantidade total de matrizes a serem recalculadas pelo método.

3.3. Avaliação Computacional

Foram implementados como artefatos retornados em cada processo de ajuste fino, arquivos contendo os ΔW resultantes para cada camada em cada bloco de transformer, assim como suas matrizes de valores singulares e avaliações sobre essas matrizes. Extraímos o rank dessas matrizes, a quantidade de valores singulares necessária para explicar 90% e 95% da matriz original e a normas associadas.

Além disso, também foi implementado um registro de métricas de hardware durante o treinamento dos modelos, com coleta ao longo da avaliação de cada época. Esse monitor reúne informações de CPU, uso de RAM do processo e do sistema, além de métricas de GPU quando disponível, combinando psutil, nvidia-smi e estatísticas do torch.cuda para capturar tanto médias quanto picos de utilização e memória.

A ideia é que os resultados se debrucem na análise desses artefatos, nos ajudando a visualizar que esses métodos, com a mesma validade da implementação clássica, aplicam mudanças no modelo que nos permitem um treinamento mais inteligente e com menor recurso computacional requerido.

4. Resultados e discussão

Para as rodadas que geraram os resultados abordados nesse trabalho foi utilizada uma NVIDIA L4 dentro do Google Colab com batch_size de 32. Buscamos aqui avaliar as implementações sob as mesmas condições e traçar uma discussão sobre os resultados que demonstre como as especificidades de cada método transparecem no seu processo de treino.

4.1. Parâmetros treináveis e uso de hardware

Uma das principais motivações para a implementação dos métodos aqui propostos é a inviabilidade do ajuste fino de modelos muito grandes. Esse problema é endereçado na implementação desses métodos pois reduzimos de forma muito notável a quantidade de parâmetros recalculados no processo, demandando menor quantidade de recursos computacionais.

Table 1: Cálculo de parâmetros treináveis por método.

Método	Número de parâmetros treináveis	Porcentagem de parâmetros treináveis (%)
Full Fine-tuning	124439808	100
LoRA	589824	0.471747
AdaLoRA	590016	0.471900

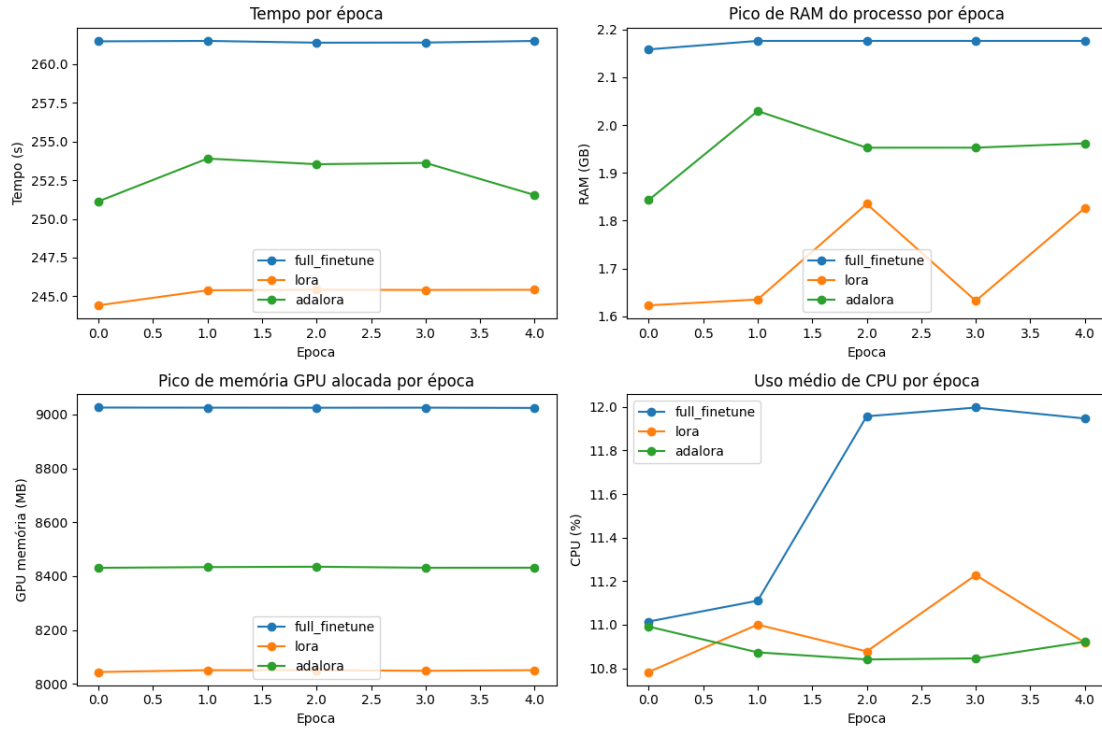


Figure 1: Uso de hardware e tempo por época em cada método implementado

Esse resultado também foi refletido no uso de RAM, GPU e CPU registrado durante o treino, essa redução nos testes não foi tão expressiva pois aqui trabalhamos com um modelo menor, mas já valida que é um resultado esperado dessas adaptações e como mostrado em [Zhang et al. (2023)] e [Hu et al. (2021)] escala de maneira muito representativa quando tratamos de modelos grandes como o GPT-3 de 175 Bilhões de parâmetros.

4.2. Declínio de loss

Apesar do AdaLoRA ter um parâmetro a mais a ser somado dentro da sua *loss*, guardamos também para esse método o que seria a sua *loss* original e aqui a usamos para avaliação comparativa.

É possível perceber na 2 que todas as implementações saem do mesmo ponto, o que reflete que estamos iniciando a partir das mesmas matrizes de pesos derivadas do GPT-2 pré-treinado. Na primeira iteração observamos que imediatamente todos os modelos tem perda significativa de *loss*, mas em níveis diferentes: enquanto o *full_finetune* tem o nível mais baixo de *loss* logo na primeira rodada, o *adalora* tem um decaimento mais suave e o método *lora* se encontra entre esses dois "extremos".

Quando retiramos o primeiro valor de *loss* calculado conseguimos ver com maior detalhe a inclinação das curvas por modelo, deixando mais explícito que o método *full_finetune* começa um possível processo de *overfitting* a partir da segunda iteração, enquanto vemos que o *adalora* e o *lora* seguem descendo, cada um em sua inclinação. Esse resultado conversa diretamente com [Aghajanyan et al. (2020)] mostrando que dar muita liberdade ao LLM no treinamento acaba possivelmente degradando mais rapidamente o seu processo de ajuste, pois os modelos de linguagem pré-treinados tem uma dimensionalidade intrínseca baixa e utilizam os graus extras de liberdade para fitar o que é ruído do dataset. Portanto, podemos também observar as limitações de *rank* das matrizes de atualização ΔW como um método de regularização para evitar o *overfitting* e ter essa como mais

uma característica dessas implementações que motiva o seu uso.

Um outro resultado interessante refletido na análise desse gráfico é a queda menos acentuada da *loss* na primeira iteração do método *adalora*, comportamento esse que se deve a definição inicial das matrizes P , Q e Λ que iniciam de forma aleatória e forçam que a *loss* inicial seja sequestrada pelo termo regularizador que as força ser ortogonal. Dessa forma, o método acaba focando bastante nas primeiras épocas em ortogonalizar essas matrizes e demora um pouco mais para focar no aprendizado da nova tarefa. Apesar disso, percebemos que o método *adalora* tem a maior inclinação da curva de *loss* mostrando que uma vez que a ortogonalidade das matrizes está resolvida, o AdaLoRA consegue de forma mais eficiente trabalhar o processo de ajuste fino do modelo.

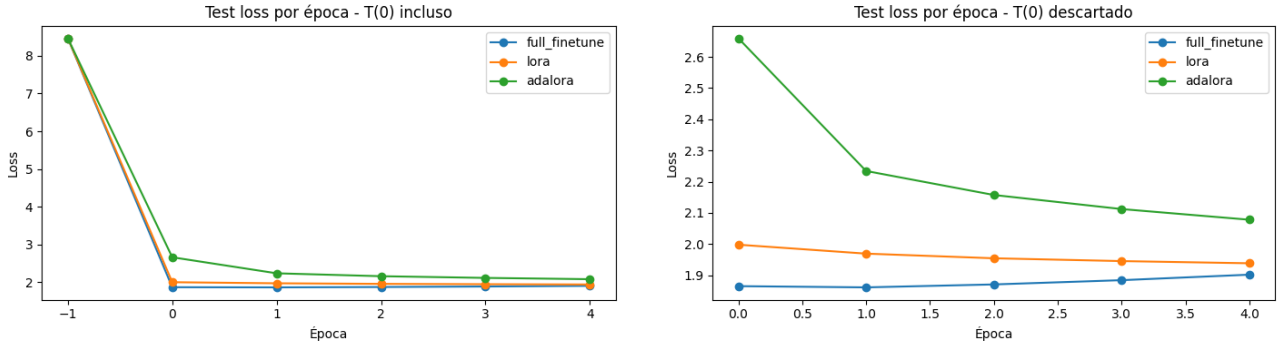


Figure 2: *Test loss* entre métodos implementados. Podemos ver que todos os métodos estão com a mesma perda antes da primeira iteração, que diminui de forma diferente para cada método na primeira iteração e nas épocas seguintes tem comportamentos característicos de sua definição matemática.

4.3. Análise algébrica das matrizes de atualização

Comprovado que o método de *fine tuning* clássico é inviável computacionalmente para modelos grandes e também pouco efetivo por estar bastante propenso ao *overfitting*, fica evidente que o AdaloRA e o LoRA são melhores escolhas para o processo. Entretanto, ainda precisamos justificar a escolha de criar um método mais complexo e que requer maior processamento que o LoRA, essa escolha se dá após conseguirmos comprovar que o *rank* estático proposto pelo LoRA também é subótimo para o cálculo das matrizes ΔW .

Com o *rank* fixo, o LoRA obriga o modelo a sempre dar o mesmo nível de importância para cada uma das camadas lineares, o que [Zhang et al. (2023)] comprova que não faz tanto sentido pois ao fazer o recálculo apenas das camadas superiores (top layers 10, 11 e 12) o modelo entrega uma performance significativamente maior do que o fine-tuning apenas das camadas inferiores (1, 2 e 3).

A tabela 2 mostra o *rank* efetivo das matrizes, calculado como aproximação contínua do posto, definida pela razão entre a "energia total" da matriz (Norma de Frobenius ao quadrado) e a energia da sua direção principal (Norma Espectral ao quadrado):

$$\text{stable_rank}(\Delta W) = \frac{\|\Delta W\|_F^2}{\sigma_1^2}.$$

Foi também calculada a quantidade de valores singulares necessários para o nível de explicabilidade de 90% e 95% de cada matriz. Esse resultado demonstra que quando permitido que o *rank* das ma-

trizes seja guiado pela importância que elas tem para o *fine-tuning*, o resultado converge para uma distribuição não uniforme do *rank* entre matrizes. Esse comportamento intensifica a característica de regularização desse método, o tornando menos propenso a sofrer *overfitting* enquanto pode proporcionar graus de liberdade para matrizes importantes na efetividade da tarefa de ajuste fino.

Table 2: Comparação dos *ranks* efetivos das atualizações de peso nas camadas de atenção do GPT-2 para full fine-tuning, LoRA e AdaLoRA.

Layer (attn.c.attn)	Full fine-tuning			LoRA			AdaLoRA		
	Stable rank	Rank 90%	Rank 95%	Stable rank	Rank 90%	Rank 95%	Stable rank	Rank 90%	Rank 95%
0	12.373869	352	469	2.511853	4	4	3.142552	4	4
1	21.363171	288	392	2.771834	4	4	2.709429	4	4
2	23.161821	292	395	2.712073	4	4	2.891301	3	3
3	49.480592	326	432	2.589969	4	4	2.449246	3	3
4	64.206663	350	457	2.747323	4	4	0.999797	1	1
5	48.537991	370	477	3.147379	4	4	2.718430	4	4
6	66.754836	372	480	2.676144	4	4	1.863167	3	3
7	65.222239	385	493	2.494931	4	4	2.541164	3	3
8	50.331039	378	487	3.148203	4	4	1.678029	2	2
9	50.054273	384	493	2.781731	4	4	1.769157	2	2
10	42.388022	384	496	2.735873	4	4	1.833540	2	2
11	14.498845	364	481	2.372519	4	4	2.027082	3	4

5. Conclusão

Este trabalho investigou a relação entre a Decomposição em Valores Singulares e técnicas de adaptação eficiente de modelos de linguagem, utilizando como estudo de caso o fine-tuning do GPT-2 com os métodos Full Fine-Tuning, LoRA e AdaLoRA.

Os resultados evidenciaram que os métodos de adaptação de baixo posto reduzem de forma expressiva a quantidade de parâmetros treináveis podendo chegar, nos métodos mais modernos, a aproximadamente 0,47% do total. Embora a diferença de consumo de recursos tenha sido limitada pelo porte relativamente pequeno do GPT-2 utilizado nos experimentos, essa redução confirma o princípio que motiva esses métodos: restringir o espaço de atualização pode tornar o ajuste fino significativamente mais viável para modelos de maior escala.

A análise das curvas de perda também indicou comportamentos distintos entre os métodos e demonstrou que a baixa dimensionalidade das atualizações não deve ser interpretada apenas como uma estratégia de economia computacional, mas também como uma forma de restringir o modelo a direções de adaptação potencialmente mais relevantes.

A análise algébrica das matrizes de atualização reforçou essa interpretação trazendo, com o AdaLoRA, uma distribuição não uniforme do rank entre as camadas: algumas matrizes concentraram sua energia em apenas um ou dois componentes, enquanto outras mantiveram três ou quatro componentes relevantes. Esse resultado confirma, na implementação desenvolvida, a hipótese de que diferentes camadas possuem diferentes necessidades de capacidade adaptativa.

Portanto, conclui-se que LoRA e AdaLoRA materializam, no contexto de modelos de linguagem, o resultado teórico de que aproximações de baixo posto podem preservar a estrutura mais relevante de uma transformação matricial com um número muito menor de graus de liberdade e demonstram como conceitos centrais da álgebra linear, em especial a SVD e as aproximações matriciais de baixo posto, possuem aplicação direta e relevante em métodos modernos de adaptação de redes neurais.

Disponibilidade do código

O código que acompanha este relatório pode ser encontrado o repositório:

<https://github.com/vitmesquita/computational-linear-algebra-lora>

Referências

- Aghajanyan, A., Zettlemoyer, L., and Gupta, S. (2020). Intrinsic dimensionality explains the effectiveness of language model fine-tuning.
- Eckart, C. and Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., and Chen, W. (2021). Lora: Low-rank adaptation of large language models. *CoRR*, abs/2106.09685.
- Merity, S., Xiong, C., Bradbury, J., and Socher, R. (2016). Pointer sentinel mixture models.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., et al. (2020). Transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:2010.11929*.
- Zhang, Q., Chen, M., Bukharin, A., Karampatziakis, N., He, P., Cheng, Y., Chen, W., and Zhao, T. (2023). Adalora: Adaptive budget allocation for parameter-efficient fine-tuning.